

# System Use Cases & User Stories

Comprehensive Specifications for the Drone Delivery Microservices Architecture

## 1. Ordering & Command Context

### Use Case 1: Place New Shipment [cite: 1]

#### USER STORY

*As a customer or system operator, I want to create a new shipment order with origin, destination, and package details so that the fulfillment process can begin.*

#### ACTORS

API Client, Order Service (Command Handler)

#### PRECONDITIONS

Valid payload specifications and routing coordinates are provided.

#### MAIN FLOW

- Client sends a POST request to `/shipments` with payload attributes [cite: 1].
- Order Service generates a unique identifier (SHP-XXXXXX) [cite: 1].
- Domain logic validates rules and emits a `ShipmentPlaced` event [cite: 4].
- Event is appended to the Event Store and projected to read models [cite: 1, 2].

## Use Case 2: Confirm Shipment & Assign Drone [cite: 1]

### USER STORY

*As a system dispatcher or mission coordinator, I want to confirm orders and bind specific drones to shipments so that physical transport can be scheduled.*

### ACTORS

Mission Service, Order Service, Dispatcher

### MAIN FLOW

- Service issues confirmation command via `/shipments/:id/confirm` [cite: 1].
- Mission Orchestrator selects an available idle drone matching weight constraints [cite: 3].
- A POST request is sent to `/shipments/:id/drone` recording the drone assignment [cite: 1, 3].
- Domain emits `ShipmentConfirmed` and `DroneAssigned` events [cite: 4].

## Use Case 3: Update Status & Audit History [cite: 1]

### USER STORY

*As an auditor or system administrator, I want to track lifecycle status changes and inspect raw domain event streams for debugging and compliance.*

### ACTORS

System Administrator, Auditor, Monitoring Tools

### MAIN FLOW

- Status updates are patched via `PATCH /shipments/:id/status` [cite: 1].
- Audit queries fetch raw event streams from `GET /shipments/:id/events` [cite: 1].
- System projects state transitions (e.g., `IN_TRANSIT`, `DELIVERED`, `CANCELLED`) [cite: 4].

## 2. Fleet & Mission Context

### Use Case 4: Orchestrate Drone Missions [cite: 3]

#### USER STORY

*As a fleet manager, I want to automatically allocate drones, track battery levels, and manage delivery flight phases so that shipments are transported efficiently.*

#### ACTORS

Mission Orchestrator, Drone Fleet, Order Service

#### PRECONDITIONS

Suitable idle drone with sufficient payload capacity exists [cite: 3].

#### MAIN FLOW

- Client requests mission creation via `POST /missions` [cite: 3].
- System changes drone status to `ON_MISSION` and initiates takeoff phase [cite: 3].
- Mission Service triggers synchronous callbacks to Order Service updating status to `IN_TRANSIT` [cite: 3].
- Upon completion (`POST /missions/:id/complete`), drone status reverts to `IDLE` [cite: 3].

## 3. Tracking & Observability Context

### Use Case 5: Track Shipment & Consume Events [cite: 4]

#### USER STORY

*As an end user, I want to query my package location, real-time timeline history, and estimated time of arrival (ETA) without impacting backend write performance.*

#### ACTORS

End Customer, Tracking Service, API Gateway

#### MAIN FLOW

- Tracking Service consumes asynchronous domain events pushed from Order Service via HTTP [cite: 1, 4].
- Internal map updates local timeline entries and calculates ETA based on status [cite: 4].
- Customer queries tracking status via `GET /tracking/:shipmentId` [cite: 4].

## Use Case 6: System Observability & Metrics Scraping [cite: 1, 3, 4, 6]

---

### USER STORY

*As a DevOps engineer, I want all microservices to expose health checks and Prometheus metrics so that cluster uptime and performance can be monitored.*

### ACTORS

Prometheus, DevOps Engineer, API Gateway

### MAIN FLOW

- Prometheus scrapes `/metrics` endpoints across all services [cite: 6].
- Liveness and dependency status are verified via `/health` endpoints [cite: 1, 3, 4].
- API Gateway enforces circuit breaker thresholds on failing upstream calls [cite: 6].